

Transitions: DS + Algorithms

Tree-based
data structures



Trees:

Dfn: A tree (in data structures) is a set of nodes which store elements in a parent-child relationship.

Any tree has a special uppermost node
↳ called the root.

Each node (except the root) has a unique parent.

Note: Not quite the same as in graph theory!

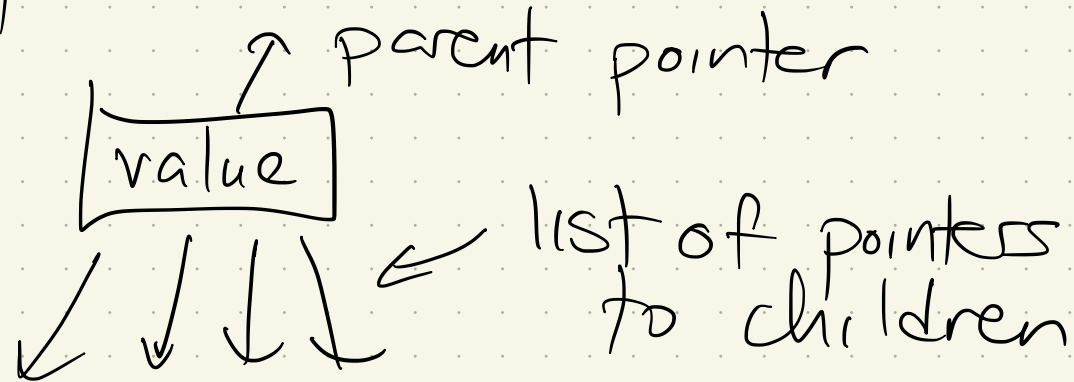
Practical examples:

Anything w/ parent-child relationship
(rather than linear)

- File systems.
- Family trees
- Parse trees for sentences or equations

A general tree implementation:

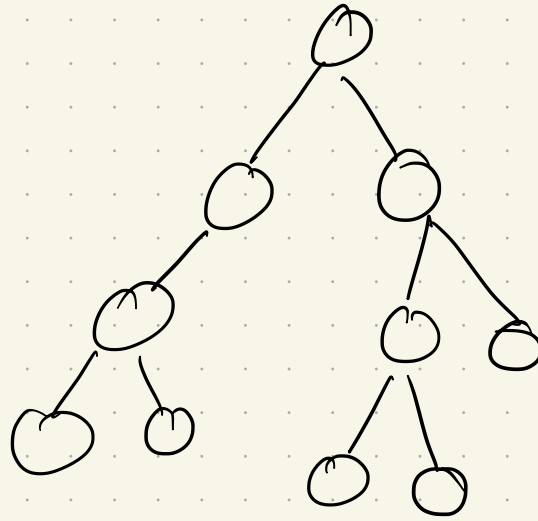
Usually pointer based



Plus global variables:

Binary Trees

- Every node has ≤ 2 children:



Full tree:

Every node has 0 or 2.

Complete tree:

Every node has 2 except perhaps lowest level which fills left to right.

Depth + Height:

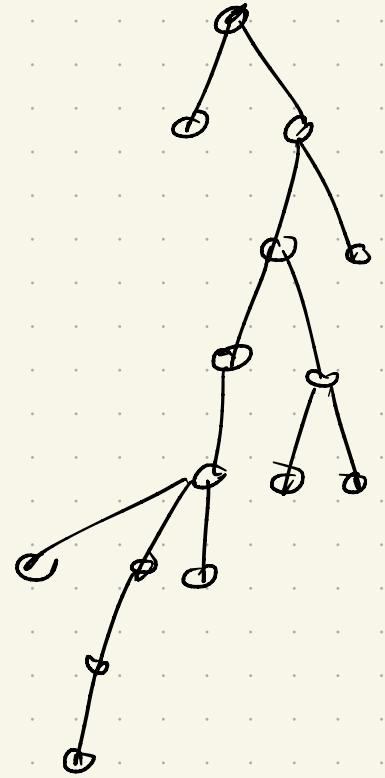
Both defined recursively.

$$\text{depth}(r) = 0$$

$$\text{depth}(v) = \text{depth}(\text{parent}(v)) + 1$$

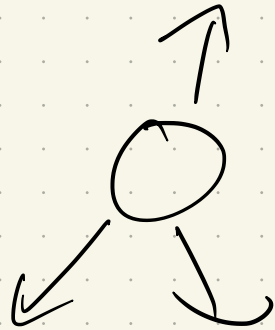
$$\text{height}(\text{leaf}) = 0$$

$$\text{height}(v) = \max \{ \text{height of } v\text{'s child} \} + 1$$

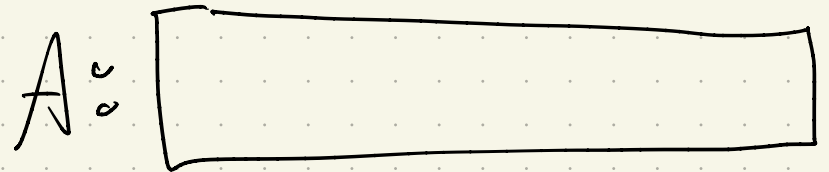
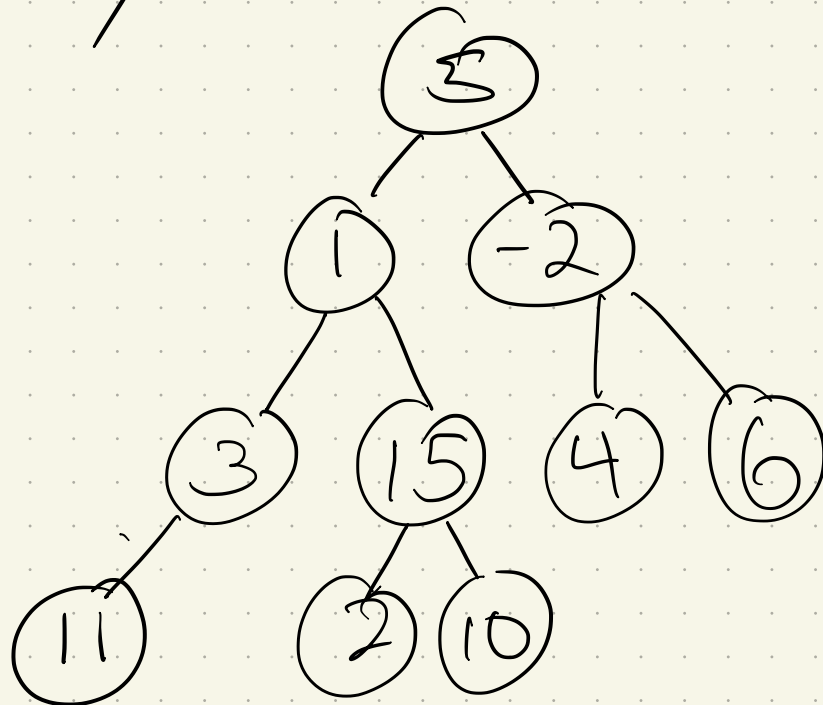


Implementation:

• Pointer based: 3 pointers



• Array based:

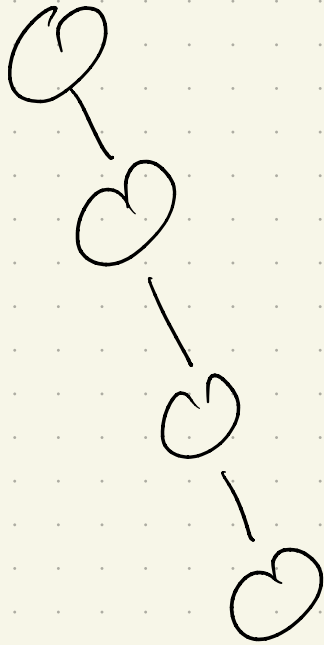


$left(v) =$
 $right(v) =$
 $parent(v) =$

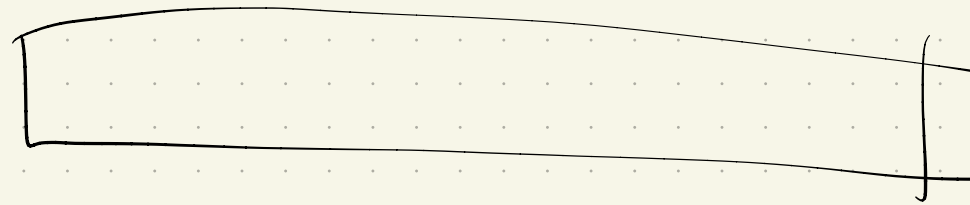
Potential downside of array:

Space!

Consider:



A:



How big?

Aside: Tree traversals

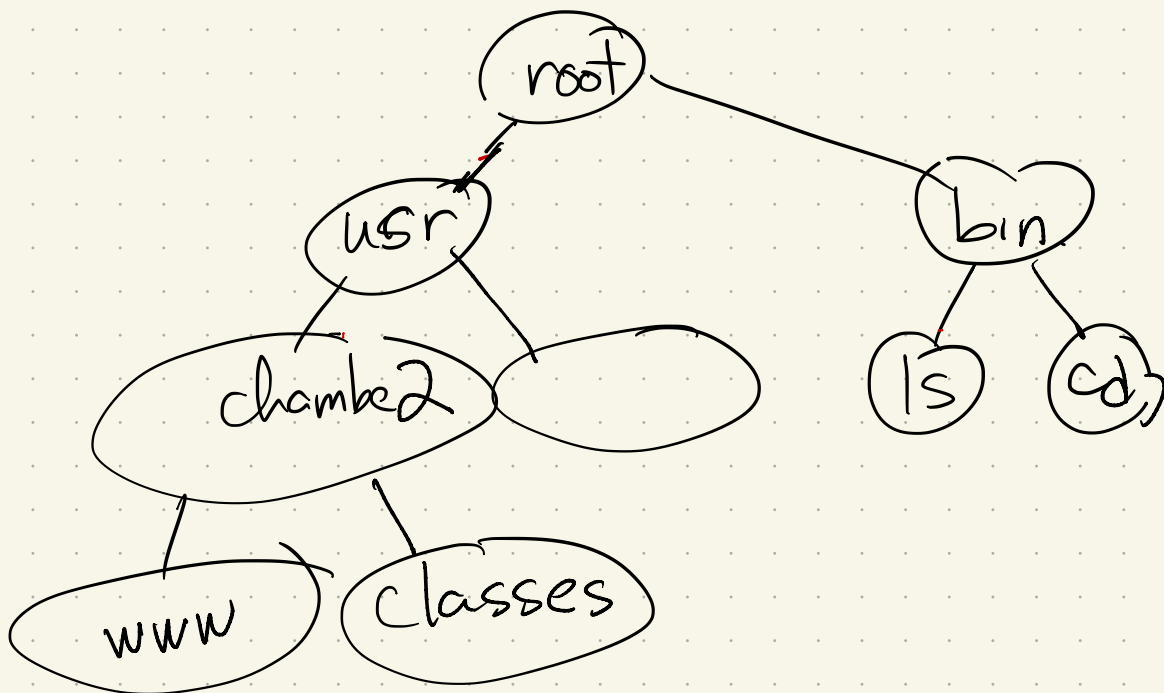
3 ways to traverse a tree:

Starting at root:

Preorder(v) =
visit v

Preorder(v $\rightarrow$ left)

Preorder(v $\rightarrow$ right)



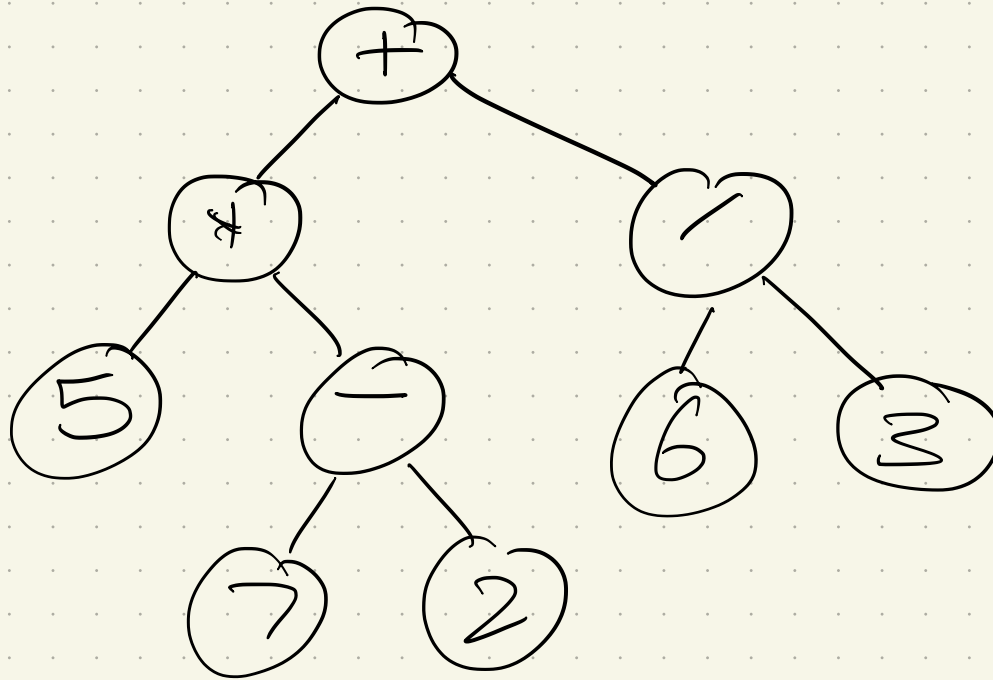
In order (v) =

Inorder (v $\rightarrow$ left)

visit v

Inorder (v $\rightarrow$ right)

Ex:

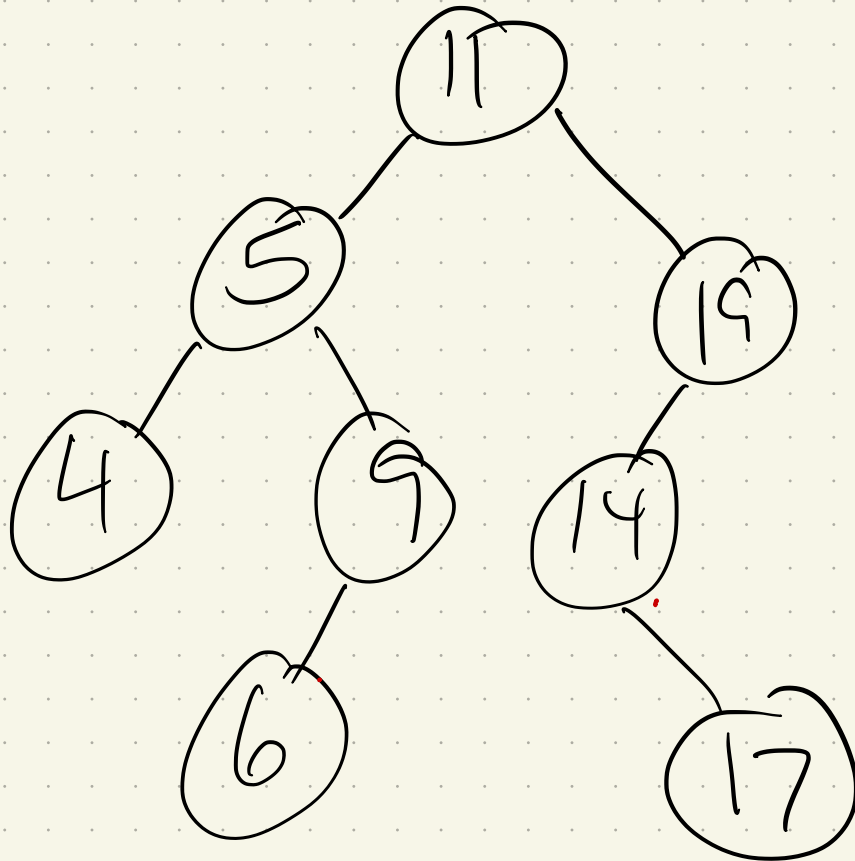


traversal:

Post order(v) =

post order($v \rightarrow \text{left}$)
post order($v \rightarrow \text{right}$)
visit v

Ex:



We'll see two tree based data structures:

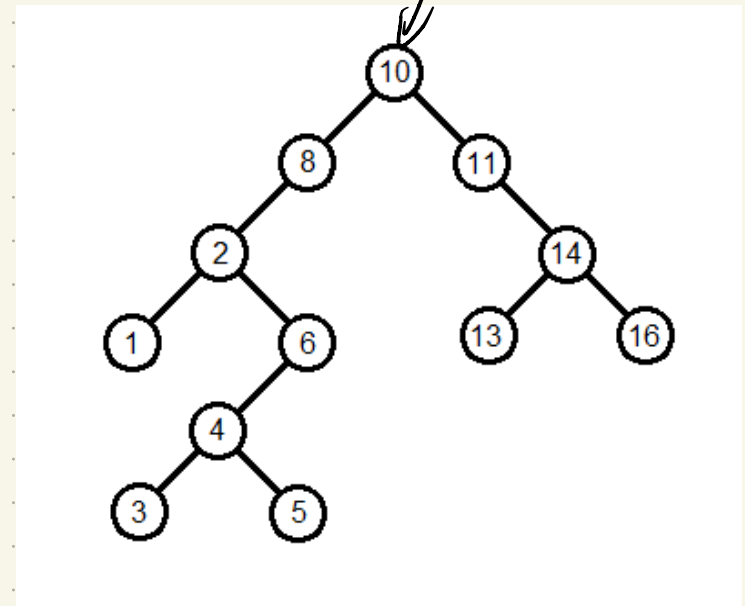
First: Binary Search Trees

A binary tree where we have a comparison operator on data, & maintain the

following: • The value at any node is $\geq$ its left child, and $<$ its right child.

Ex:

Assuming this data, how can we search for a value?



Pseudocode:

Comparison to lists/vectors:

Linear search:

Binary search:

In tree:

Reading:

More on BSTs, & well as
"Balanced" BSTs.

Then: Heaps & Hashing next
week